

By following modern system engineering practices simulation of reactive systems is no longer necessary

▶ True

▶ False [click here for detail](#)

The state transition diagram

▶ depicts relationships between data objects

▶ depicts functions that transform the data flow

▶ indicates how data are transformed by the system

▶ [indicates system reactions to external events click here for detail](#)

Control flow diagrams are

▶ [needed to model event driven systems. Click here for detail](#)

▶ required for all systems

▶ used in place of data flow diagrams

▶ useful for modeling user interfaces

A change becomes ----- because of close presence of data and functions

▶ [Localized \(page 81\)](#)

▶ Private

- ▶ Global
- ▶ Accessible

A complex System evolves from a

- ▶ smaller system page 83
- ▶ medium system
- ▶ bigger system
- ▶ non of the given

Coupling is a measure of -----of a module or component.

- ▶ Independent (Page 71)
- ▶ Dependent
- ▶ Closeness
- ▶ All of the given

Software Maintenance phase involves

- ▶ Debugging
- ▶ Adding new features
- ▶ making changes
- ▶ all of the given click here for detail

The hardest single part of building a software system is deciding precisely -----
--- to build.

▶ what (Page 17)

▶ How

▶ When

▶ Why

Interaction Diagrams depict the ----- Behavior of the system

▶ Static

▶ Dynamic (Page 106)

▶ Active

▶ Non of the above

In Sequence Diagrams the time required by the receiver object to process the
message is denoted by an -----

▶ Activation Box (Page 108)

▶ Simple Box

▶ Arrow

▶ Non of the above

A poorly designed interface can cause a user to make catastrophic errors is one of the motivations for GUI.

▶ True (Page 62)

▶ False

Identifying System Features include -----

▶ log important information

▶ conduct business

▶ analyze business results

▶ all of the above (Page 98)

Establishing responsibilities for objects includes

▶ Generalization Relationships

▶ Specialization Relationships

▶ all of the above (Page 86)

▶ identifying Association relationships

The Object Oriented approach

▶ improves the reusability of codes. (Page 83)

▶ makes objects less independent.

- ▶ increases testing time.
- ▶ complicates mapping of a system model to an implementation model.

Most software continues to be custom built because

- ▶ Software is easier to build without using someone else's components.
- ▶ Off the shelf software components are not commonly available [Click here for detail](#)
- ▶ Component reuse is common in the software world
- ▶ Reusable components are too expensive to use

Which of the items listed below is not one of the software engineering layers?

- ▶ Tools
- ▶ Process
- ▶ Manufacturing
- ▶ Methods

A change becomes ----- because of close presence of data and functions

- ▶ Localized (page 81)
- ▶ Private
- ▶ Global

- ▶ Accessible

A process which does not take input is called

▶ miracle process Miracle": Process has output, but no input Click here for detail

- ▶ core process
- ▶ secondary process
- ▶ all of the above

Data Store notation in DFDs presents.

- ▶ Data input
- ▶ Data output
- ▶ Data input and Data output (Page 54)
- ▶ non of the given

Questions:

Question No: 1 (Marks: 3) Define Asynchronous Messages and Synchronous Messages.

Answer:- (Page 109)

Asynchronous Messages:-

Asynchronous messages are “signals,” denoted by a half arrow. They do not block the caller. Asynchronous messages typically perform the following actions:

Create a new thread

Create a new object

Communicate with a thread that is already running Synchronous Messages:-

Synchronous messages are “call events” and are denoted by the full arrow. They represent nested flow of control which is typically implemented as an operation call. In case of a synchronous message, the caller waits for the called routine to complete its operation before moving forward.

Question No: 2 (Marks: 5) Law of balancing act in software

Answer:- (Page 7) The Balancing Act! Software Engineering is actually the balancing act. You have to balance many things like cost, user friendliness, Efficiency, Reliability etc. You have to analyze which one is the more important feature for your software is it reliability, efficiency, user friendliness or something else. There is always a trade-off among all these requirements of software. It may be the case that if you try to make it more user-friendly then the efficiency may suffer. And if you try to make it more cost-effective then reliability may suffer. Therefore there is always a trade-off between these characteristics of software. These requirements may be conflicting. For example, there may be tension among the following:

Cost vs. Efficiency

Cost vs. Reliability

Efficiency vs. User-interface

A Software engineer is required to analyze these conflicting entities and tries to strike a balance.

Question No: 3 (Marks: --) HOW DO YOU DETERMINE THAT AN OBJECTIVE BELONGS TO CERTAIN CLASS?

Answer:- (Page 85) The basic unit of object oriented design is an object. An object can be defined as a tangible entity that exhibits some well defined behavior. The structure and behavior of similar objects are defined in their common class. A class specifies an interface and defines an implementation.

Question No: 4 (Marks: --) What is meant by "System's Static View"?

Answer:- (Software engineering ---Page 608) Static view of semantic classes. Requirements are assessed and classes are extracted (and represented) as part of the analysis model. Static view of attributes. Every class must be explicitly described. The attributes associated with the class provide a description of the class. Static view of relationships. Objects are "connected" to one another in a variety of ways. The analysis model must represent these. Static view of behaviors. The relationships just noted define a set of behaviors that accommodate the usage scenario (use-cases) of the system

Question No: 5 (Marks: --) What is behavior driven perceive of an objective?

Answer:- (Page 85) Behavior is how an object acts and reacts in terms of its state changes and message passing. The behavior of an object is completely defined by its actions. A message is some action that one object performs upon another in order to elicit a reaction. The operations that clients may perform upon an object are called methods.

VU Insider